JPA, Part 3

M:N and 1:1 associations, inheritance.

Learning goals

1. Learn to implement M:N and 1:1 association types with JPA.
2. Learn the annotations and options for mapping inheritance relationships.

M:N



#	Nimi	Tietotyyppi
1	Employee_empid	INT
2	Project_projectid	INT

- A many-to-many association type always generates a link table into the DB.
  - Every record in the link table contains references to both parties of the association.
  - The aggregate of the referencing fields constitutes the primary key (a composite primary key).
- The same database-level implementation is used in spite of the chosen navigability of the association.
- Let's consider the depicted situation in the case where the navigation direction is from Employee to Project.

Example: Employee

Employee.java

```
@Entity
public class Employee  {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int empId;
    private String firstName;
    private String lastName;
    private double salary;
    @ManyToMany(fetch=FetchType.EAGER)
    private List<Project> projects = new ArrayList<Project>();
```

Project.java

```
@Entity
public class Project {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int projectId;
    private String name;
    private int budget;
```

- In a uni-directional association, the @ManyToMany annotation is attached to the class that provides the navigability to the other class.
- That class becomes automatically the owner of the relationship.
  - When an employee is saved, the corresponding link table record is inserted at the same time.

Example: saving

EmpApp.java

```
Employee emp1 = new Employee("Mary", "Smith", 3600.0);
Employee emp2 = new Employee("Urpo", "Koski", 2600.0);

Project pr1 = new Project("Database review", 8400);
Project pr2 = new Project("Server acquisition", 42000);
Project pr3 = new Project("Customer service improvement", 16500);

emp1.getProjects().add(pr1);
emp1.getProjects().add(pr2);
emp2.getProjects().add(pr2);
emp2.getProjects().add(pr3);

dao.saveProject(pr1);
dao.saveProject(pr2);
dao.saveProject(pr3);
dao.saveEmployee(emp1);
dao.saveEmployee(emp2);
```

Example: saving

```
Hibernate: insert into Project (budget, name) values (?, ?)
Hibernate: insert into Project (budget, name) values (?, ?)
Hibernate: insert into Project (budget, name) values (?, ?)
Hibernate: insert into Employee (firstName, lastName, salary) values (?, ?, ?)
Hibernate: insert into Employee_Project (Employee_empId, projects_projectId) values (?, ?)
Hibernate: insert into Employee_Project (Employee_empId, projects_projectId) values (?, ?)
Hibernate: insert into Employee (firstName, lastName, salary) values (?, ?, ?)
Hibernate: insert into Employee_Project (Employee_empId, projects_projectId) values (?, ?)
Hibernate: insert into Employee_Project (Employee_empId, projects_projectId) values (?, ?)
```

company.employee: 2 r Next Show al

empId	firstName	lastName	salary
1	Mary	Smith	3 600
2	Urpo	Koski	2 600

company.project: 3 row: Next Show all

projectId	budget	name
1	8 400	Database review
2	42 000	Server acquisition
3	16 500	Customer service improvement

company.employee_pro Next Sh

Employee_empId	projects_projectId
1	1
1	2
2	2
2	3

Example: loading

controller

```
Employee emp = dao.loadEmployee(2);
System.out.println(emp.getFirstName());
for (Project pr : emp.getProjects()) {
    System.out.println(pr.getName());
}
```

Note: In Spring Boot, the dao variable refers to the service object

```
Hibernate: select employee0_.empId as empId1_1_0_, employee0_.firstName as firstNam2_1_0_, empl
Urpo
Server acquisition
Customer service improvement
```

- Note that eager loading must be set on.
  - In lazy loading, retrieval of employee's projects' names would target an uninitialized proxy, yielding a LazyInitializationException.
  - In @ManyToMany associations, lazy loading is on by default.

Bidirectional association

Employee.java

```
@Entity
public class Employee  {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int empId;
    private String firstName;
    private String lastName;
    private double salary;
    @ManyToMany(fetch=FetchType.EAGER)
    private List<Project> projects = new ArrayList<Project>();
```

Project.java

```
@Entity
public class Project {

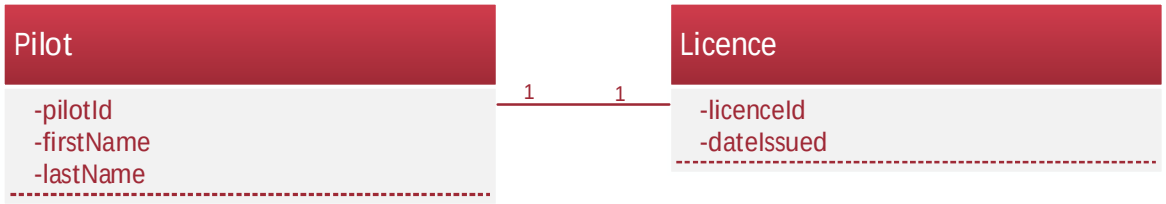
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int projectId;
    private String name;
    private int budget;
    @ManyToMany(mappedBy="projects")
    private List<Employee> employees = new ArrayList<Employee>();
```

- In a bidirectional M:N association, both classes must contain a data structure instance variable (List or Set).
- A @ManyToMany annotation is added to both classes.
- One of the classes must be declared owner of the association (the other class gets a mappedBy definition).
  - Failing to do that, Hibernate would generate two link tables, one of them being redundant.

Supplementary fields in a link table

- Let's consider a situation where the developer want to insert additional fields in the link table.
  - E.g. a timestamp would be such a field: it could describe when the association between the entities has been established,
- Since JPA does not support 'extra' fields in the link tables, it is necessary to implement a Java language class for the link table entries
  - In the class for the link table entries, one adds a @ManyToOne annotation for both linked entities.
  - The composite primary key is implemented with @IdClass or @EmbeddedId-annotations. In this case, a dedicated primary key class needs to be written.
    - See e.g. https://www.objectdb.com/java/jpa/entity/id#Composite_Primary_Key_.

1:1



- In JPA, @OneToOne annotation marks a one-to-one association.
- One-to-one associations are implemented by foreign keys at the database level.
  - This is essentially the same as for 1:M associations.
  - Multiple values of a foreign key can be prohibited by a UNIQUE constraint in the CREATE TABLE statement in the database generation script.
- Again, the relationship can be uni-directional or bi-directional.
  - Ownership is specified by the mappedBy attribute, just as with other association types.
- The membership can be made mandatory by specifying @OneToOne(optional=false).
  - This can be done for other association types as well.

DB-level implementation options

- Five implementation strategies at database level:
  1. A foreign key in the PILOT table refers to the LICENCE table.
  2. A foreign key in the LICENCE table refers to the PILOT table.
  3. Shared primary key value in both tables, one refers to the other.
  4. A common table.
  5. A link table (a 'wrong' solution from the DB design perspective.)

Basic implementation: @OneToOne

- In the most straightforward implementation, a @OneToOne annotation is added to the either party of the one-to-one association.
  - This party becomes the owner of the association.
  - The foreign key will be added to the DB table generated from that class.
- The following slide depicts a situation where the Licence class is the owner.
  - Alternatively, Pilot could be chosen as the owner. The implementation would be analogous.

Basic implementation: @OneToOne

Pilot.java

```
@Entity
public class Pilot {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int pilotId;
    private String firstName;
    private String lastName;
```

Licence.java

```
@Entity
public class Licence {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int licenceId;
    @Temporal(value=TemporalType.DATE)
    private Date dateIssued;
    @OneToOne(optional=false)
    private Pilot pilot;
```

Basic implementation: @OneToOne

```
Hibernate: insert into Pilot (firstName, lastName) values (?, ?)
Hibernate: insert into Licence (dateIssued, pilot_pilotId) values (?, ?)
```

company.pilot: 1 rows total

pilotId	firstName	lastName
1	Anna	Smith

company.licence: 1 rows total Next

licenceId	dateIssued	pilot_pilotId
1	2019-04-14	1

Bi-directional @OneToOne

- In a bi-directional 1:1 association, one party is selected as the owner.
  - Otherwise, Hibernate would maintain two foreign keys, one in each table. That would be redundant.
- Both classes are annotated with @OneToOne.
  - A `mappedBy` attribute will be added to the class that is not the owner.

Shared primary key value in both tables

- In the previous example, both the Pilot and the Licence classes had their own, independent primary keys.
- In an alternative solution, they can share the primary key values. Thus:
  - pilotId is the primary key in the PILOT table.
  - The pilotId identifier in the LICENCE table acts as both the primary key and a foreign key that referes to the PILOT table,
- As a consequence, one avoids updating two independent key sequencies, when one of them does not have any independent purpose.
- The key constraint for the LICENCE table dictates that there must always be a related pilot for each licence, i.e. the pilotId value must not be NULL.

Shared primary key value in both tables

Pilot.java

```
@Entity
public class Pilot {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int pilotId;
    private String firstName;
    private String lastName;
```

Licence.java

```
@Entity
public class Licence {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int licenceId;
    @Temporal(value=TemporalType.DATE)
    private Date dateIssued;
    @JoinColumn
    @OneToOne(optional=false)
    @MapsId
    private Pilot pilot;
```

Shared primary key value in both tables

controller

```
Dao dao = new Dao();

Pilot pi1 = new Pilot("Anna", "Smith");
Licence li1 = new Licence(new Date(System.currentTimeMillis()), pi1);

dao.savePilotAndLicence(pi1, li1);
```

Note: In Spring Boot, the dao variable refers to the service object

pilotId	firstName	lastName
1	Anna	Smith

company.licence: 1 rows total

dateIssued	pilot_pilotId
2019-04-14	1

- The service object should persist both the pilot and the licence objects.
- Note that in this approach the Pilot object must be in EntityManager´s scope whilst saving a Licence object.
  - Thus, a dedicated service method for saving both entities is written.
- In the LICENCE table, the pilot_pilotId field acts both as a primary key and a foreign key.

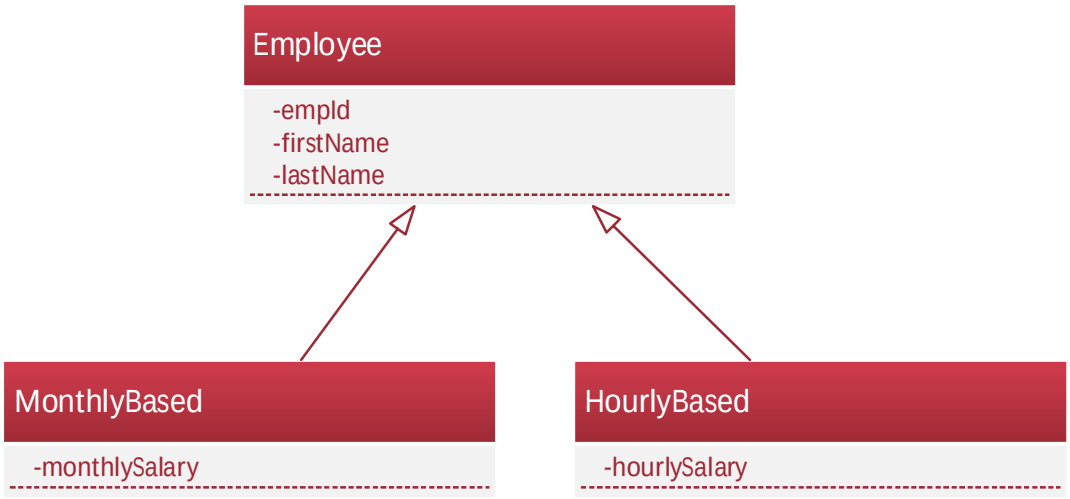
Common table

- At times, 1:1 association is implemented with a single table at the database level.
- The data of the both parties are incorporated into a common table.
- E.g. a pilot's licence data is stored in the PILOT table, among the pilot's information.
- This is implemented with @Embedded- ja @Embeddable-annotaatioiden avulla.
  - The party that defines the primary key, must not be optional.
  - See, e.g. http://www.thejavageek.com/2014/01/25/jpa-embeddable-annotation/

Link table

- A 1:1 association can be implemented by a link table just like in the M:N solution.
- From a DB design perspective, the link table is not necessary.
- When both fields in the link table have the `unique=true` option set, that *de facto* defines a 1:1 association.

Inheritance and ORM



- The inheritance relationship poses a challenge for object-relational mapping.
  - The relationship does not have a straightforward counterpart in the relational model.
- Let's consider a situation where an employee is paid either on a monthly or hourly basis (not both).

Implementation strategies

- JPA provides three implementation strategies for inheritance:
  1. A common table for the entire class hierarchy (InheritanceType.SINGLE_TABLE).
  2. Distinct tables for each subclass and the superclass. The object's subclass data is joined with the superclass data (InheritanceType.JOINED).
  3. Each concrete subclass produces a table of its own that contains all instance variable values, including those inherited from the superclass (InheritanceType.TABLE_PER_CLASS).

Example: superclass

Employee.java

```
@Entity
@Inheritance
public abstract class Employee  {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int empId;
    private String firstName;
    private String lastName;
```

- @Inheritance annotation is attached to each superclass.
- At this point, one of the three implementation strategies can be specified.

Example: subclasses

MonthlyBased.java

```
@Entity
public class MonthlyBased extends Employee {

    private double monthlySalary;
```

HourlyBased.java

```
@Entity
public class HourlyBased extends Employee {

    private double hourlySalary;
```

Strategy 1: single table for entire class hierarchy

Employee.java

```
@Entity
@Inheritance(strategy=InheritanceType.SINGLE_TABLE)
public abstract class Employee  {
```

Single table strategy

company.employee: 2 rows total

Next | Show all | Sorting

DTYPE	empId	firstName	lastName	hourlySalary	monthlySalary
MonthlyBased	1	Mary	Smith	(NULL)	3 600
HourlyBased	2	Urpo	Koski	17,45	(NULL)

- The instance variables from the subclasses are stored in the same table than those of the superclass.
- A discriminator field (by default named DTYPE) holds the subclass membership information.
- The solution requires that NULL values are allowed in both salary fields.
- This is the default strategy.

Strategy 2: joined tables

Employee.java

```
@Entity
@Inheritance(strategy=InheritanceType.JOINED)
@DiscriminatorColumn
public abstract class Employee  {
```

company.employee: 2 rows total (approximately)

DTYPE	empId	firstName	lastName
MonthlyBased	1	Mary	Smith
HourlyBased	2	Urpo	Koski

company.monthlybased: 1 rows t (approximately)

monthlySalary	empId
3 600	1

company.hourlybased: 1 rows t (approximately)

hourlySalary	empId
17,45	2

- Each subclass generates a new table, including the superclass.
- In this strategy, the discriminator field is not auto-generated.
  - @DiscriminatorColumn annotation generates it, if desired.

Strategy 3: a table per concrete class

Employee.java

```
@Entity
@Inheritance(strategy=InheritanceType.TABLE_PER_CLASS)
public abstract class Employee  {
```

company.monthlybased: 1 rows total

empId	firstName	lastName	monthlySalary
1	Mary	Smith	3 600

company.hourlybased: 1 rows total

empId	firstName	lastName	hourlySalary
2	Urpo	Koski	17,45

- Both subclasses produce tables, one each.
- The subclass-specific tables also contain fields for inherited instance variables.
- In this strategy, the primary keys cannot be generated by GenerationType.IDENTITY strategy, as all tables need to share a common sequence.
  - Use GenerationType.TABLE key generation strategy instead.

company.hibernate_sequences: 1 rows total

sequence_name	next_val
default	3

Loading data

- In loading subclass/superclass data, a new programming challenge emerges: the subclass memberships of an object is known only after the retrieval.
  - Such data is stored in the database, so it is inherently not ’hard-codeable’.
- Solution: retrieve the object into a variable of the (abstract) superclass.
- After the retrieval, the object can be downcast to its true class.

Which is the best inheritance strategy?

- Whatever the inheritance strategy, the exact constraints of the inheritance relationship cannot be reflected in the table structure.
  - E.g. the constraint ”an object must be member of exactly one subclass of the abstract superclass” is not expressable by relational database constraints.
- The TABLE_PER_CLASS strategy can be inefficient if the query target the superclass data.
  - E.g. querying an employee based on family name may generate two SELECT statements in the DB.
    - One for monthly-based, and another for hourly-based.
- SINGLE_TABLE strategy is space-inefficient if the subclasses have many instance variables.
  - Lots of NULL values required.